

SOFTWARE REQUIREMENTS SPECIFICATION

ChargeHub

Smart EV Charging & Trip Planner

Find. Book. Charge. Drive Further — an intelligent companion for the modern EV owner.



Document Version: 1.0

Document Type: IEEE-Style SRS

Status: Draft for Evaluation

SUBMITTED BY / PROJECT DETAILS

Student Name : _____

University Roll No. / ID : _____

Class, Section & Semester : _____

Course / Subject : _____

Team Members (if group project) : _____

Project Guide / Supervisor : _____

Department : _____

Institution / College Name : _____

Submission Date : _____

Table of Contents

1. Introduction4

1.1 Purpose4

1.2 Document Conventions4

1.3 Intended Audience and Reading Suggestions4

1.4 Project Scope4

1.5 References4

2. Overall Description5

2.1 Product Perspective5

2.2 Product Functions (Summary)5

2.3 User Classes and Characteristics5

2.4 Operating Environment5

2.5 Design and Implementation Constraints5

2.6 Assumptions and Dependencies6

2.7 Apportioning of Requirements6

3. System Features6

3.1 Find Nearby Charging Stations6

3.2 View Charger Availability & Speed6

3.3 Compare Charging Prices7

3.4 Book Charging Slots7

3.5 Navigation to Charging Stations7

3.6 EV Trip Planner7

3.7 Charging Cost Estimation8

3.8 Favourites & Charging History8

3.9 User Account & Vehicle Profile8

4. External Interface Requirements9

4.1 User Interfaces9

4.2 Hardware Interfaces9

4.3 Software Interfaces9

4.4 Communications Interfaces9

5. Data Requirements10

5.1 Conceptual Data Entities 10

5.2 High-Level Entity Relationships 10

5.3 Sample REST API Endpoints (Planning Reference) 10

6. Non-Functional Requirements 11

7. System Architecture Overview 11

7.1 High-Level Architecture 11

7.2 Trip Planner Algorithm (Conceptual) 12

7.3 Technology Stack (Suggested) 12

8. Use Case Summary 12

9. Software Testing Plan 13

9.1 Testing Scope 13

9.2 Testing Levels 13

9.3 Sample Test Cases 13

10. Project Plan & Deliverables 14

10.1 Suggested Development Phases 14

10.2 Future Enhancements (Out of Current Scope) 14

11. Appendices 14

Appendix A — Glossary 15

Appendix B — Assumption Log 15

Appendix C — Revision History 15

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for **ChargeHub**, a Smart EV Charging & Trip Planner application. It is intended to provide a complete, unambiguous, and testable description of the system to be developed, and serves as a reference for the student development team, the project guide/evaluator, and any future contributors to the project. This document follows the general structure recommended by the IEEE 830 / IEEE 29148 standards for Software Requirements Specifications, adapted for the scope of a college software engineering project.

1.2 Document Conventions

Requirement identifiers follow the pattern **FR-XX** for functional requirements and **NFR-XX** for non-functional requirements. Priority levels are expressed as **High**, **Medium**, or **Low**. The keywords *shall*, *should*, and *may* are used respectively to denote mandatory, recommended, and optional behaviour, consistent with standard requirements-engineering practice.

1.3 Intended Audience and Reading Suggestions

- **Students / Developers** — read Sections 3–6 for functional detail, data design, and API planning.
- **Faculty / Evaluators** — read Sections 1, 2, and 7–9 for scope, feasibility, and testing coverage.
- **UI/UX Designers** — read Sections 4.4 and 5 for interface and workflow requirements.
- **Project Guide** — read Section 2 (Overall Description) and Section 9 (Project Plan) for milestones.

1.4 Project Scope

ChargeHub is a mobile-first application that helps electric vehicle (EV) owners locate nearby charging stations, check real-time charger availability and pricing, reserve charging slots in advance, and plan long-distance trips with automatically suggested charging stops based on the vehicle's battery level and route. The system promotes sustainable transportation by reducing "range anxiety" and making EV charging as convenient as refuelling a conventional vehicle. The project is scoped to be realistically achievable by a small student team within a single academic semester, while remaining architecturally sound enough to extend into a production-grade product in later phases.

1.5 References

Ref.	Document / Standard	Description
[1]	IEEE 29148:2018	IEEE Standard for Requirements Engineering — structure reference
[2]	IEEE 830-1998	Recommended Practice for Software Requirements Specifications
[3]	Open Charge Point Interface (OCPI)	Reference protocol concepts for charger data exchange
[4]	Google Maps Platform Docs	Reference for maps, geocoding, and routing API design
[5]	Material Design / HIG	UI/UX guideline reference for mobile app design

2. Overall Description

2.1 Product Perspective

ChargeHub is a new, standalone, self-contained mobile/web application. It is not a modification of an existing system. It integrates with third-party mapping and geolocation services (for maps, geocoding, and navigation) and, in later phases, with charging-network partner APIs for live station data. For the academic prototype, station and charger data may be simulated/seeded in the project's own database to demonstrate full functionality without requiring live commercial API access.

2.2 Product Functions (Summary)

- Locate nearby EV charging stations using GPS and map-based search.
- View charger type, availability status, and charging speed (AC/DC, kW rating).
- Compare charging prices across nearby stations.
- Book / reserve a charging slot at a selected station and time.
- Get turn-by-turn navigation to the chosen charging station.
- Plan multi-stop trips with charging-stop recommendations based on battery range.
- Estimate charging cost before starting a session.
- Save favourite stations and view past charging history.
- Manage user profile, vehicle details, and notification preferences.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
EV Owner (Primary User)	Registered user who searches for stations, books slots, and plans trips.	Low–Medium
Station Operator / Admin	Manages station details, charger status, and pricing (Phase 2 scope).	Medium
System Administrator	Maintains user accounts, monitors system health, resolves disputes.	High
Guest User	Can browse nearby stations without logging in; limited functionality.	Low

2.4 Operating Environment

- **Client:** Android/iOS mobile application (or responsive web app) running on standard consumer devices.
- **Server:** Cloud-hosted backend (e.g., a Node.js/Django/Spring Boot REST API) deployed on a Linux-based server or PaaS.
- **Database:** Relational database (e.g., PostgreSQL/MySQL) for structured data such as users, stations, and bookings.
- **External Services:** Maps & geolocation API, push-notification service, and (future) payment gateway.

2.5 Design and Implementation Constraints

- The prototype must run on limited/free-tier cloud and map API quotas typically available to students.
- Development timeline is constrained to a single academic semester (approx. 12–16 weeks).
- The team size is small (2–4 members); scope is intentionally kept realistic rather than exhaustive.
- Real-time live data from commercial charging networks may not be accessible; simulated data will be used where necessary, with the architecture designed to accept real data feeds in later phases.

2.6 Assumptions and Dependencies

- Users have an internet-connected smartphone with GPS enabled.
- A third-party maps/geocoding API (e.g., Google Maps or OpenStreetMap) will be available for development.
- Charging station data (location, connector type, pricing) is either sourced from open datasets or seeded manually for demonstration purposes.
- Payment processing is out of scope for the core prototype and is simulated (a real gateway can be integrated in a future phase).

2.7 Apportioning of Requirements

Some requirements marked as **Phase 2** throughout this document (e.g., live payment integration, operator dashboards, and third-party network API integration) are not required for the initial academic submission but are documented here to demonstrate architectural forward-planning.

3. System Features

This section describes each major feature of ChargeHub along with its associated functional requirements, priority, and a representative user story.

3.1 Find Nearby Charging Stations

User Story: *As an EV owner, I want to see charging stations near my current location so that I can quickly find a place to charge.*

ID	Requirement	Priority
FR-01	The system shall detect the user's current location via device GPS (with manual entry as fallback).	High
FR-02	The system shall display nearby charging stations on an interactive map within a configurable search radius.	High
FR-03	The system shall allow filtering of stations by connector type, charging speed, and distance.	Medium
FR-04	The system shall display a list view as an alternative to the map view.	Low

3.2 View Charger Availability & Speed

User Story: *As an EV owner, I want to check whether a charger is available and how fast it charges before travelling there.*

ID	Requirement	Priority
FR-05	The system shall display real-time (or last-known) availability status for each charger: Available, In Use, or Out of Service.	High
FR-06	The system shall display the connector type (Type 2, CCS, CHAdeMO, etc.) and power rating (kW) of each charger.	High
FR-07	The system shall estimate approximate charging time based on the vehicle's battery capacity and charger speed.	Medium

3.3 Compare Charging Prices

User Story: *As a cost-conscious EV owner, I want to compare prices across nearby stations so I can choose the most economical option.*

ID	Requirement	Priority
FR-08	The system shall display the price per kWh (or per session/minute) for each station.	High
FR-09	The system shall allow sorting of search results by price, distance, or availability.	Medium

3.4 Book Charging Slots

User Story: *As an EV owner, I want to reserve a charging slot in advance so I don't waste time waiting when I arrive.*

ID	Requirement	Priority
FR-10	The system shall allow a registered user to book an available slot at a selected station for a chosen date and time.	High
FR-11	The system shall prevent double-booking of the same charger for overlapping time slots.	High
FR-12	The system shall allow users to view, modify, or cancel an existing booking.	Medium
FR-13	The system shall send a booking confirmation and a reminder notification before the scheduled slot.	Medium

3.5 Navigation to Charging Stations

User Story: *As an EV owner, I want turn-by-turn directions to my selected charging station.*

ID	Requirement	Priority
FR-14	The system shall provide navigation/directions to a selected station using an integrated maps service.	Medium
FR-15	The system shall display estimated travel time and distance to the selected station.	Medium

3.6 EV Trip Planner

User Story: *As an EV owner planning a long trip, I want the app to suggest charging stops along my route based on my battery level.*

ID	Requirement	Priority
FR-16	The system shall allow the user to input a start location, destination, and current battery percentage.	High
FR-17	The system shall calculate estimated driving range from the current battery level and the vehicle's efficiency profile.	High
FR-18	The system shall recommend one or more charging stops along the route when the destination exceeds the vehicle's remaining range.	High
FR-19	The system shall allow the user to accept, skip, or replace a suggested charging stop.	Medium
FR-20	The system shall recalculate the route and suggested stops if the user deviates from the planned path.	Low

3.7 Charging Cost Estimation

User Story: *As an EV owner, I want to know the estimated cost of a charging session before I begin.*

ID	Requirement	Priority
FR-21	The system shall calculate an estimated cost based on charger price, target battery percentage, and vehicle battery capacity.	Medium
FR-22	The system shall display a running/actual cost estimate once a charging session has started (simulated for the prototype).	Low

3.8 Favourites & Charging History

User Story: *As a frequent user, I want to save my favourite stations and review my past charging sessions.*

ID	Requirement	Priority
FR-23	The system shall allow users to mark/unmark a station as a favourite for quick access.	Medium
FR-24	The system shall maintain a history of the user's past bookings and charging sessions.	Medium
FR-25	The system shall allow the user to view total energy consumed and total amount spent over a selected period.	Low

3.9 User Account & Vehicle Profile

User Story: *As a user, I want to manage my account and register my EV's details so the app can tailor recommendations to my vehicle.*

ID	Requirement	Priority
FR-26	The system shall allow users to register and log in using email/password (or OAuth, e.g., Google Sign-In).	High
FR-27	The system shall allow a user to add and edit vehicle details: make, model, battery capacity (kWh), and connector type.	High
FR-28	The system shall allow users to update profile information and notification preferences.	Low

4. External Interface Requirements

4.1 User Interfaces

- **Home / Map Screen** — map view centred on the user's location with nearby station pins.
- **Station Detail Screen** — chargers, availability, pricing, ratings, and a “Book Slot” action.
- **Booking Screen** — date/time picker, connector selection, and confirmation summary.
- **Trip Planner Screen** — origin/destination input, battery slider, and a route map with suggested stops.
- **Profile Screen** — user details, saved vehicles, favourites, and charging history.
- **Login / Sign-Up Screens** — standard authentication flows with input validation and error messaging.

Wireframes and high-fidelity mockups are maintained separately in the UI/UX design deliverable and should follow a clean, map-centric layout consistent with modern navigation apps, using clear status colours (green = available, amber = busy, red = out of service).

4.2 Hardware Interfaces

The application relies on the host mobile device's GPS/location sensor for positioning. No direct hardware interface to the EV or charger hardware is required for the academic-scope prototype; charger status is read from the application database rather than from physical charger telemetry.

4.3 Software Interfaces

Interface	Purpose	Notes
Maps & Geocoding API	Display maps, geocode addresses, compute routes and travel time.	e.g., Google Maps Platform / OpenStreetMap + OSRM
Authentication Provider	User sign-up/sign-in (optional social login).	Firebase Auth / OAuth 2.0
Push Notification Service	Booking confirmations and reminders.	Firebase Cloud Messaging
Application Database	Persistent storage for users, stations, bookings, history.	PostgreSQL / MySQL
Payment Gateway (Phase 2)	Real payment capture for charging sessions.	Razorpay / Stripe (future integration)

4.4 Communications Interfaces

The mobile/web client communicates with the backend server over HTTPS using a RESTful JSON API. Real-time availability updates may use periodic polling for the prototype, with an upgrade path to WebSockets for true real-time updates in a later phase.

5. Data Requirements

5.1 Conceptual Data Entities

Entity	Key Attributes	Notes
User	user_id, name, email, password_hash, phone	One user may own multiple vehicles
Vehicle	vehicle_id, user_id (FK), make, model, battery_kwh, connector_type	Linked to User
Station	station_id, name, address, latitude, longitude, operator	Has many Chargers
Charger	charger_id, station_id (FK), connector_type, power_kw, status, price_per_kwh	Linked to Station
Booking	booking_id, user_id (FK), charger_id (FK), start_time, end_time, status	Prevents double-booking
ChargingSession	session_id, booking_id (FK), energy_kwh, cost, timestamp	Historical record
Favourite	favourite_id, user_id (FK), station_id (FK)	Many-to-many mapping
TripPlan	trip_id, user_id (FK), origin, destination, battery_pct, suggested_stops	Stores planner results

5.2 High-Level Entity Relationships

A **User** owns one or more **Vehicles** and may create multiple **Bookings**. Each **Station** contains one or more **Chargers**; each **Charger** can have many **Bookings** over time, but only one active booking per time slot. A **TripPlan** references a sequence of recommended **Stations** as charging stops. This structure directly maps to a normalized relational schema suitable for third normal form (3NF) implementation.

5.3 Sample REST API Endpoints (Planning Reference)

Method	Endpoint	Description
POST	/api/auth/register	Register a new user account
POST	/api/auth/login	Authenticate a user and issue a session/JWT token
GET	/api/stations/nearby?lat&lng;&radius;	Retrieve nearby charging stations
GET	/api/stations/{id}	Retrieve full detail for a single station
POST	/api/bookings	Create a new charging slot booking
GET	/api/bookings/{userId}	Retrieve a user's bookings and history

Method	Endpoint	Description
DELETE	/api/bookings/{id}	Cancel an existing booking
POST	/api/trip-planner	Submit trip parameters and receive suggested charging stops
POST	/api/favourites	Add or remove a favourite station

6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Station search results shall load within 3 seconds under normal network conditions.
NFR-02	Usability	A first-time user shall be able to locate and view a nearby station within 3 taps of opening the app.
NFR-03	Reliability	The booking system shall guarantee no double-booking of a single charger for the same time slot.
NFR-04	Availability	The backend service shall target 99% uptime during the demonstration/evaluation period.
NFR-05	Security	User passwords shall be stored using a secure one-way hash (e.g., bcrypt); all API traffic shall use HTTPS.
NFR-06	Scalability	The architecture shall support horizontal scaling of the backend API layer in future phases.
NFR-07	Maintainability	The codebase shall follow a layered architecture (presentation, business logic, data access) to ease maintenance.
NFR-08	Portability	The client application shall be responsive/cross-platform, supporting both Android and iOS or a mobile-responsive web build.
NFR-09	Compatibility	The system shall function correctly on the latest two major versions of Android and iOS.

7. System Architecture Overview

7.1 High-Level Architecture

ChargeHub follows a standard three-tier architecture appropriate for a student project while remaining extensible for production use:

- 1 **Presentation Layer** — Mobile app (e.g., Flutter/React Native) or responsive web front-end, responsible for maps, forms, and user interaction.
- 2 **Application / Business Logic Layer** — REST API server handling authentication, station search, booking logic, and trip-planning algorithms.

- 3 **Data Layer** — Relational database storing users, vehicles, stations, chargers, bookings, and history; integrates with external Maps API for geospatial queries.

7.2 Trip Planner Algorithm (Conceptual)

The trip planner uses a simplified range-based stop-selection approach suitable for a student project scope:

- 1 Compute the vehicle's estimated range from current battery percentage and battery capacity (kWh) using an average efficiency factor (km/kWh).
- 2 Fetch the route from origin to destination via the Maps/Directions API.
- 3 If the destination distance exceeds the estimated range, identify candidate charging stations within a corridor along the route.
- 4 Select the charging stop(s) that best balance detour distance, charger availability, and remaining range at arrival (e.g., choosing a stop near 20–30% remaining battery as a safety margin).
- 5 Return the ordered list of recommended stops to the client for display and user confirmation.

7.3 Technology Stack (Suggested)

Layer	Suggested Technology	Rationale
Front-end (Mobile)	Flutter or React Native	Single codebase for Android & iOS; student-friendly ecosystem
Front-end (Web, optional)	React.js	Widely documented; component-based UI for map dashboards
Back-end API	Node.js (Express) or Django REST Framework	Rapid REST API development with strong community support
Database	PostgreSQL (with PostGIS for geo-queries) or MySQL	Reliable relational storage; PostGIS aids proximity search
Maps & Routing	Google Maps Platform or OpenStreetMap + OSRM	Mature APIs for maps, geocoding, and directions
Hosting	Render / Railway / AWS Free Tier / Firebase	Free or low-cost tiers suitable for student deployment

8. Use Case Summary

The table below summarises the primary use cases corresponding to the system features in Section 3, intended to support use-case diagram creation during design.

Use Case	Primary Actor	Preconditions	Main Success Outcome
Search Nearby Stations	EV Owner	Location services enabled	List/map of nearby stations displayed
View Station Detail	EV Owner	Station exists in system	Charger status, price, and speed shown

Use Case	Primary Actor	Preconditions	Main Success Outcome
Book Charging Slot	EV Owner (registered)	User logged in; charger available	Slot reserved and confirmed
Cancel Booking	EV Owner (registered)	Active booking exists	Booking cancelled; slot released
Plan Trip	EV Owner	Origin, destination, battery % provided	Route with charging stops returned
Estimate Cost	EV Owner	Station and target charge % selected	Estimated cost displayed
Manage Favourites	EV Owner (registered)	User logged in	Station added/removed from favourites
View Charging History	EV Owner (registered)	User logged in	Past sessions and spend displayed
Register / Login	EV Owner / Guest	None	Authenticated session established

9. Software Testing Plan

9.1 Testing Scope

Testing shall cover functional correctness of all requirements listed in Section 3, key non-functional properties from Section 6, and integration points with external map and notification services.

9.2 Testing Levels

Level	Focus	Example
Unit Testing	Individual functions/modules (e.g., range calculation, cost estimator).	Verify range = battery_kWh × efficiency
Integration Testing	Interaction between API, database, and external map service.	Booking creation updates charger status correctly
System Testing	End-to-end user flows across the full application.	Search → View → Book → Navigate flow
Acceptance Testing	Validation against SRS requirements with the project guide/evaluator.	Demo walkthrough against FR-01 to FR-28
Usability Testing	Ease of use for first-time users.	Time-to-first-booking measurement

9.3 Sample Test Cases

Test ID	Requirement	Test Description	Expected Result
TC-01	FR-01/FR-02	Open app with location enabled near a seeded station.	Nearby stations appear on map within radius.
TC-02	FR-11	Attempt to book an already-booked charger for the same time slot.	System rejects the booking with a conflict message.
TC-03	FR-16–FR-18	Enter origin, destination beyond vehicle range, and battery %.	At least one charging stop is suggested along the route.
TC-04	FR-21	Select a station and a target charge percentage.	Estimated cost is displayed and matches price × energy calculation.
TC-05	FR-26	Register with an invalid/duplicate email.	System shows a validation error and rejects the registration.

10. Project Plan & Deliverables

10.1 Suggested Development Phases

Phase	Duration (Indicative)	Key Deliverables
Phase 1 — Planning & Design	Weeks 1–3	SRS (this document), ER diagram, wireframes, tech-stack finalisation
Phase 2 — Core Backend & Database	Weeks 4–6	Auth API, station/charger APIs, database schema implementation
Phase 3 — Core Front-end	Weeks 7–9	Map screen, station detail, booking flow implementation
Phase 4 — Trip Planner & Cost Estimation	Weeks 10–12	Route + stop-suggestion logic, cost calculator
Phase 5 — Testing & Polish	Weeks 13–14	Test execution, bug fixes, UI polish
Phase 6 — Documentation & Presentation	Weeks 15–16	Final report, demo video/slides, viva preparation

10.2 Future Enhancements (Out of Current Scope)

- Live integration with real charging-network operator APIs (OCPI-based).
- Real payment gateway integration for actual transaction processing.
- Operator/admin web dashboard for managing stations and pricing.
- Predictive charger-availability using historical usage patterns (ML-based).
- In-app community reviews and ratings for charging stations.

11. Appendices

Appendix A — Glossary

Term	Definition
EV	Electric Vehicle
kWh	Kilowatt-hour, a unit of energy used to measure battery capacity and consumption
kW	Kilowatt, a unit of power used to describe charger speed
CCS / CHAdeMO / Type 2	Common EV charging connector standards
Range Anxiety	Driver concern about running out of battery charge before reaching a charging point
OCPI	Open Charge Point Interface — a protocol for exchanging charging data between platforms
SRS	Software Requirements Specification

Appendix B — Assumption Log

All assumptions listed in Section 2.6 apply throughout this document unless explicitly overridden within a specific requirement. Any change to a core assumption (e.g., availability of a live charging-network API) should trigger a revision of the affected requirements and a version update to this SRS.

Appendix C — Revision History

Version	Date	Description	Author
1.0	—	Initial SRS draft prepared for academic submission.	Project Team